

AI-Based Job Recommendation System Using Skill Extraction from Resumes

Final Project Paper – Data Mining

202503_Datamining_CSCI_6443_82

By

Rohith Sunkara (G22074927)

Virat Karthikeya Koramati (G47022370)

Sai Srinivas G (G20957666)

Abstract

Matching resumes to suitable job roles is a time-consuming and error-prone task for both recruiters and job seekers, largely due to the high volume of applications and the inconsistent way skills and job titles are described. This project presents a skill-based job recommendation system that automatically analyzes resumes, suggests relevant job roles, and identifies missing skills.

The system is trained on more than 32,000 real-world job postings collected from Naukri.com. Job titles and associated skill sets are cleaned and standardized through normalization and canonicalization to improve consistency across the dataset. Skills are represented using Term Frequency–Inverse Document Frequency vectorization, allowing the model to learn common skill patterns associated with different job categories. A stochastic gradient descent-based logistic regression model is used for job role prediction due to its efficiency and scalability.

Resumes submitted in PDF format are first converted into text using pdfplumber. The extracted content is then analyzed using llm to identify relevant skills and provide clear, human-readable explanations for missing competencies. Experimental results show strong predictive performance, with the correct job role consistently appearing among the top recommendations. Overall, this work demonstrates how combining lightweight machine learning techniques with generative artificial intelligence can streamline resume screening and provide practical, actionable insights in real-world hiring workflows.

1. Introduction

This project develops a machine learning based job recommendation model that predicts suitable job roles from a given set of skills. The goal is to learn how skills map to job roles using historical job posting data and to apply this knowledge to recommend relevant roles and highlight missing skills for new inputs.

The approach models job roles as output classes and represents required skills as textual features. By training on a large collection of real job postings, the model learns which combinations of skills commonly appear together within specific job roles. Rather than comparing resumes to individual job descriptions, the system directly predicts job roles based on these learned skill patterns, making the process more scalable and consistent.

The paper describes the dataset used for training, the preprocessing steps applied to reduce noise and fragmentation across job titles, and the feature representation and learning methodology. It

also examines how the trained model behaves during prediction. The evaluation section reports both classification and ranking performance, followed by a discussion of observed limitations and areas where the model can be improved.

2. Dataset Description

The dataset used in this project was collected from the Naukri.com job portal and consists of individual job postings. Each entry corresponds to a single job listing and includes details such as the job title, company information, experience requirements, location, and a list of required skills. While the dataset contains several attributes, this study focuses primarily on job titles and required skills, as these fields are the most relevant for learning relationships between skills and job roles.

In total, the dataset includes more than 32,000 job postings. Job titles are provided as free text and show substantial variation due to differences in naming conventions, seniority levels, and role descriptions across companies. Required skills are also listed in an unstructured format, often separated by new lines or commas, with inconsistent capitalization and wording.

To illustrate the structure of the data, some sample job postings are shown below.

	job_post	required_skills
149	Data Engineer - GoLang (Professional 2)	[go lang, github, jenkins, angular, nosql, css...
190	Senior Software Engineer-SSE - AWS Glue ETL de...	[redshift, glue etl, debugging, aws glue, lamb...
531	Data Engineer - Senior	[it skills, python, cloud, dba, oracle, aws, p...
544	Data Engineer Business Intelligence	[framework manager, report studio, cognos 11 v...
564	Senior Technical Lead (Data Engineer)	[excel, recruitment, powerpoint, vendor manage...
...	...	...
32333	Scouting for Big Data Developers Night Shift Only	[data analysis, interpersonal skills, excel, t...
32423	Data Engineering Application Developer	[business process, data analysis, paas, consul...
32462	ETL Developer	[written communication, informatica, rdms, et...
32471	ETL Developer	[written communication, informatica, rdms, et...
32563	Informatica Big Data Edition Application Devel...	[communication, analytical, sql, business proc...

Figure 1: Example job postings from the dataset illustrating job titles and associated required skills for the Data Engineer role.

This example demonstrates how each entry in the dataset links a job title to a corresponding set of required skills. Although the dataset shows considerable variation in how job titles and skills are expressed, every posting follows this basic structure. This consistency allows the model to learn meaningful relationships between recurring skill patterns and specific job roles.

2.1 Exploratory Data Analysis

Exploratory data analysis was carried out to understand the structure of the dataset and identify key challenges before preprocessing and model development.

An initial review of the job title field showed a large amount of variation, with more than fifteen thousand unique titles in the raw data. Many titles differ only in wording, seniority level, technology names, or added location and hiring-related text. Because of this, the label distribution is highly imbalanced. A small number of common job roles appear very often, while many titles occur only a few times. This makes direct multi-class classification difficult and highlights the need for job title normalization and ranking-based evaluation rather than strict single-label prediction.

The required skills field showed a similar pattern. A small group of general skills appears in many job postings, while a large number of specialized skills appear only rarely. This long-tail distribution suggests that simple keyword matching would not work well. Instead, it supports the use of TF-IDF feature representation, which helps give more importance to role-specific skills and reduces the influence of very common ones.

3. Data Cleaning and Preprocessing:

Data cleaning was a major technical challenge in this project due to the highly inconsistent and unstructured nature of the raw dataset collected from Naukri.com. Both job titles and required skills were provided as free-text fields and showed substantial variation in formatting, spelling, and content. To address these issues, a multi-stage preprocessing pipeline was developed to ensure that the data was consistent, interpretable, and suitable for supervised learning.

3.1 Handling Missing Values and Invalid Records

The original dataset contained 32,738 job postings. Records that were missing a valid job title or a required skills field were removed, since both fields are essential for training a supervised classification model. In addition, entries where the required skills field was empty, null, or otherwise unusable were discarded. This filtering step ensured that every remaining instance included both a valid target label and meaningful input features.

3.2 Cleaning and Normalizing Skill Descriptions

The required skills field exhibited wide variability, including newline-separated lists, comma-separated values, hyphenated compound skills, CamelCase identifiers, and partially

unstructured text. To standardize these representations, a structured cleaning pipeline was applied.

First, basic text normalization was performed by converting all skill text to lowercase, removing special characters, and trimming excess whitespace. The text was then split using multiple delimiters, including commas, newline characters, hyphens, and slashes, to handle different formatting styles consistently. This ensured that skill lists such as “python, sql”, “python\nsql”, “machine-learning”, and “html/css” were tokenized in a uniform manner.

To preserve semantic meaning, commonly used multi-word skills such as machine learning, deep learning, data analysis, and object oriented programming were retained as complete phrases rather than split into individual words. A CamelCase splitting step was also applied to transform identifiers such as MachineLearning and PowerBI into their spaced equivalents.

Finally, duplicate entries and spelling variations were resolved through normalization rules. For example, different representations such as Javascript, ReactJS, and Nodejs were standardized to javascript, react js, and node js. After preprocessing, each job posting was represented by a clean and semantically meaningful list of skills suitable for feature extraction and vectorization.

3.3 Job Title Normalization and Canonicalization

Job titles in the dataset were highly noisy and included variations related to seniority levels, domain-specific phrasing, vendor technologies, and appended location or recruitment-related text. Examples include differences such as senior and junior indicators, numeric suffixes, technology-specific titles, and phrases like immediate joiner or urgent requirement.

To reduce this variability, job titles were first normalized by removing seniority indicators, numeric levels, location names, and recruitment-specific additions. Following this, a rule-based canonicalization process was applied using keyword pattern matching to group similar titles into standardized job role categories.

As a result, more than 15,000 unique job titles were mapped into 13 canonical job roles, including data scientist, data engineer, data analyst, machine learning engineer, software engineer, frontend engineer, mobile engineer, quality assurance engineer, devops engineer, cloud architect, security engineer, product manager, project manager, and an other category. This step reduced label complexity by over 99 percent, transforming an otherwise impractical classification problem into a manageable learning task.

This canonicalization process is central to the model’s ability to generalize across related roles and to learn stable associations between skills and job categories in a high-dimensional feature space.

3.4 Consistency Between Training and Inference

The same preprocessing pipeline used during training was applied to skills extracted from user resumes during inference. This ensured consistency between the training and prediction data distributions, prevented feature mismatches, and supported reliable model behavior when deployed within the application.

4. Feature Representation

To enable supervised learning on textual skill data, the cleaned skill lists were converted into numerical feature representations using Term Frequency–Inverse Document Frequency vectorization. This method transforms unstructured skill text into a structured feature space that captures how important each skill is across job postings.

4.1 TF-IDF Vectorization

TF-IDF assigns a weight to each skill based on two factors: how often the skill appears within a single job posting and how rare the skill is across the full dataset. Skills that appear frequently within a posting but less often across all postings receive higher weights, while very common skills are down-weighted.

Each job posting is represented as a sparse feature vector, where each dimension corresponds to a unique skill or skill phrase. Skills that are shared across many roles receive lower importance, while more specialized skills contribute more strongly to the representation. This allows the model to focus on role-specific technical signals rather than generic skills that appear across many job categories.

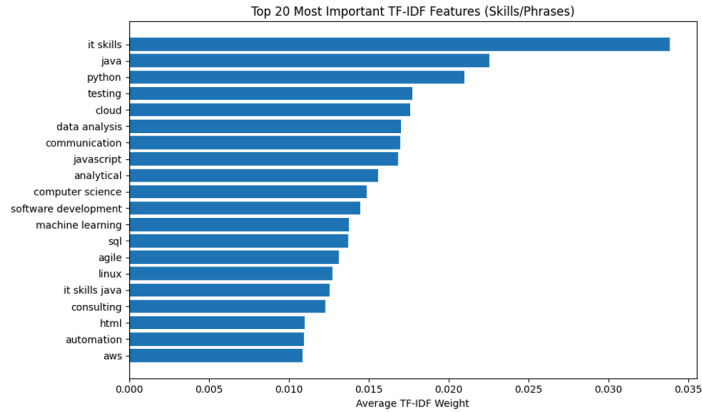


Figure 2: Distribution of high-weight skill features learned through TF-IDF vectorization.

4.2 N-gram Modeling

To preserve the meaning of multi-word technical skills, TF-IDF was configured to include n-grams ranging from one to three words. This allows the model to capture individual skills as well as commonly occurring phrases such as machine learning, data engineering, and object oriented programming. Without this configuration, such phrases would be split into separate words, leading to a loss of semantic information.

Including n-grams improves the model's ability to distinguish between job roles that may share individual terms but differ in how those terms are combined. For example, while the word learning may appear in many roles, the full phrase machine learning is much more indicative of roles focused on applied machine learning.

4.3 High-Dimensional Sparse Representation

The resulting TF-IDF feature space is high-dimensional and sparse, consisting of thousands of possible skill features, with each job posting activating only a small subset of them. This reflects the reality that most job postings list only a limited number of relevant skills.

Such sparse representations are well suited for linear classification models designed for large-scale text data. They allow efficient learning and inference without requiring dense embeddings or deep neural networks, making them a practical choice for this application.

5. Model Architecture and Training

The job recommendation task is formulated as a multi-class classification problem in which each job posting is assigned to a single canonical job role based on its associated skill set. Given the high-dimensional and sparse nature of the TF-IDF feature representation derived from skill text, a linear classification approach was selected to achieve a balance between computational efficiency, scalability, and predictive performance.

5.1 Model Structure

The proposed model follows a linear classification architecture that is well suited for sparse, high-dimensional text data. The input consists of cleaned and normalized skill text, which is transformed into numerical feature vectors using Term Frequency–Inverse Document Frequency representation. This process produces a sparse feature space in which each dimension corresponds to an individual skill or a multi-word skill phrase captured through n-gram modeling.

A multi-class linear classifier trained using stochastic gradient descent with logistic loss is applied to these TF-IDF feature vectors. For each canonical job role, the model learns a distinct set of weight parameters that represent the relative importance of different skills for that role. During inference, the model computes a score for each job role by taking the dot product between the input feature vector and the corresponding role-specific weight vector. These scores are then converted into probabilities.

The model outputs a probability distribution over all canonical job roles, allowing roles to be ranked according to their predicted relevance. This ranking-based output is more appropriate than a single-label prediction in real-world hiring and career guidance scenarios, where a given skill profile may reasonably correspond to multiple job roles.

5.2 Model Selection

The system employs a logistic regression classifier trained using stochastic gradient descent. This model optimizes the logistic loss function and is well suited for large-scale text classification tasks involving sparse feature representations.

Linear models trained with stochastic gradient descent offer several advantages in this setting. They scale efficiently to high-dimensional feature spaces, generalize well when applied to TF-IDF features, require relatively low memory overhead, and converge quickly even on large datasets. These properties make the model a practical and robust choice for learning associations

between skills and job roles without relying on computationally expensive deep neural network architectures. In addition, the linear structure of the model supports interpretability, enabling inspection of learned weight coefficients to identify the most influential skills for each job role.

5.3 Training Configuration

The dataset was divided into training and testing subsets using an eighty to twenty split. The TF-IDF vocabulary was learned exclusively from the training data, and the same vectorization process was applied to the test set to maintain consistency in feature representation and prevent information leakage.

During training, the model iteratively learns weight parameters that capture associations between individual skill features and canonical job role classes. Regularization was applied to reduce the risk of overfitting and improve generalization. Optimization proceeded using stochastic gradient descent until convergence criteria were met.

5.4 Prediction and Output Interpretation

For a given input skill vector, the trained model produces a probability distribution over all job role categories. These probabilities are used to rank job roles, and the highest-ranked roles are returned as recommendations.

This ranking-based prediction strategy is particularly well suited to job recommendation tasks, as it reflects the inherent overlap and ambiguity between related job roles. Rather than enforcing a single definitive prediction, the model provides a flexible set of relevant role suggestions, resulting in more informative and realistic career guidance.

6. Evaluation Metrics and Results

To evaluate the performance of the proposed job recommendation model, multiple evaluation metrics were used. Since the task involves predicting job roles from a large set of possible classes and producing ranked recommendations, both standard classification metrics and ranking-based measures were considered.

6.1 Evaluation Metrics

Accuracy was used to measure the proportion of job postings for which the model correctly predicted the canonical job role. While accuracy provides a general indication of prediction

correctness, it has limitations in multi-class settings, particularly when the class distribution is imbalanced.

To account for this, the weighted F1 score was also computed. This metric balances precision and recall while weighting each class according to its frequency in the dataset. As a result, the weighted F1 score offers a more reliable measure of overall performance across both common and less frequent job roles.

Given that the system is intended as a recommendation tool rather than a strict single-label classifier, Top-K accuracy was used as a primary evaluation metric. Top-K accuracy measures whether the true job role appears within the top K predicted roles. This metric closely reflects real-world usage, where presenting several relevant role suggestions is often more useful than returning a single prediction.

6.2 Experimental Results

The trained model was evaluated using accuracy, weighted F1 score, and Top-5 accuracy to measure both prediction quality and ranking performance. The model achieved an accuracy of 0.601 and a weighted F1 score of 0.563, showing that it was able to learn useful patterns between skills and canonical job roles, even with class imbalance and overlap between similar roles.

The most important result was the Top-5 accuracy of 0.972. This means that the correct job role almost always appeared within the top five predictions. This is especially meaningful for a job recommendation system, where several roles may fit a single skill profile and ranked suggestions are more helpful than a single forced choice.

Overall, the results show that while predicting one exact job role is difficult due to shared skill requirements across roles, the ranking-based approach works very well. The model consistently reduces the problem to a small set of relevant roles, making it useful for tasks such as resume screening and career guidance.

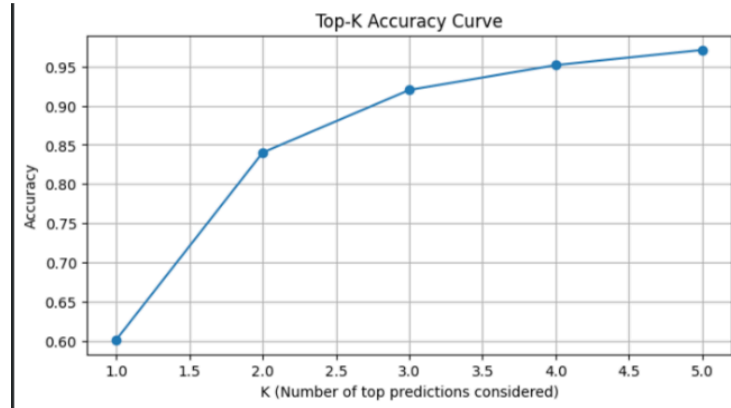


Figure 3: Top-K accuracy curve demonstrating improved recommendation performance as K increases.

6.3 Observations and Analysis

A closer analysis of the results shows that the model performs best on well-defined technical roles such as software engineer, data engineer, and data analyst, where skill requirements are relatively consistent. Performance is lower for roles grouped under the other category, which tend to include a wide range of responsibilities and less standardized skill sets.

Overall, the evaluation confirms that combining TF-IDF-based skill representation with a linear classifier trained using stochastic gradient descent provides an effective and scalable solution for skill-based job role recommendation.

7. Discussion and Limitations

The results indicate that the proposed model is effective in learning meaningful relationships between skills and job roles and in generating relevant job recommendations. The strong Top-K accuracy shows that the correct job role is consistently ranked among the top predictions, which aligns well with real-world recommendation scenarios where multiple roles may reasonably match a given skill profile.

One important observation is the distinction between job match percentage and skill match percentage. The job match score reflects how closely a user's overall skill profile aligns with the learned skill distribution of a job role, while the skill match score captures the direct overlap with the most influential skills associated with that role. As a result, a role may display a high skill match but a lower job match score, particularly when the role represents a broad category composed of multiple sub-roles with varying skill requirements.

A key design decision in this work was the canonicalization of job titles. The original dataset contained approximately twelve to fifteen thousand unique job titles, many of which differed only in wording, seniority level, or contextual additions. Training a classifier directly on this highly fragmented label space would not be feasible, especially given the high-dimensional and sparse nature of the feature representation. To address this challenge, all job titles were mapped to a reduced set of approximately fifteen to twenty canonical job role categories. While this approach significantly improves model stability, generalization, and scalability, it also reduces granularity. As a result, the system predicts broader job families rather than highly specific job titles.

The system has additional limitations. The TF-IDF representation captures surface-level lexical patterns but does not model deeper semantic relationships between related skills expressed using different terminology. The job title canonicalization process relies on rule-based pattern matching, which can occasionally lead to misclassification in cases where titles are ambiguous or poorly formatted. Moreover, the model does not explicitly incorporate factors such as experience level, industry specialization, or temporal changes in skill demand, all of which can influence job suitability.

Overall, these limitations reflect deliberate trade-offs between scalability, interpretability, and predictive precision. The current design prioritizes robust learning and practical deployment in large-scale job recommendation settings, while leaving clear opportunities for future refinement through richer role taxonomies and more semantically expressive representations.

8. System Implementation and Application Layer

The trained job recommendation model was integrated into an interactive application to support real-time user input and result interpretation. The application was developed using Streamlit, which provides a lightweight, web-based interface for resume submission and job role recommendations.

Users can either paste resume text directly into the application or upload resumes in PDF format. For PDF uploads, the system uses `pdfplumber` to extract raw textual content from the document. This raw resume text cannot be used directly as model input because it is unstructured.

To transform unstructured resume text into a structured list of skills, the system employs a large language model accessed through the LangChain framework. A prompt-based extraction strategy is used to identify relevant technical, analytical, and soft skills present in the resume. The extracted skills are then normalized using the same preprocessing pipeline applied during model training, ensuring consistency between training and inference.

The normalized skill representation is passed to the trained classification model, which produces a ranked list of predicted job roles along with corresponding match scores.

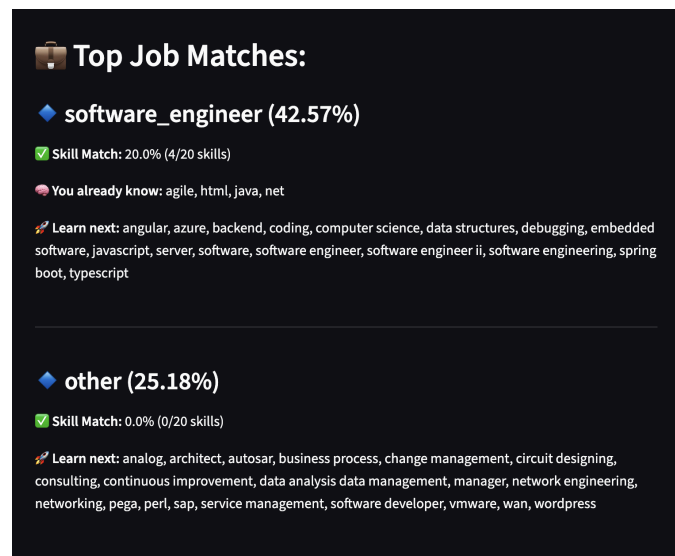


Figure 4: This figure shows the system’s ranked job role recommendations generated from a user’s resume.

In addition to role prediction, the system conducts skill gap analysis by comparing the skills extracted from a user’s resume with the most influential skills associated with each predicted job role. These role-specific skills are identified based on the highest-weighted features learned by the classification model. Skills that are strongly associated with a role but missing from the user’s profile are flagged as gaps. To improve interpretability, a language model is used to generate brief, clear explanations describing why each missing skill is important for the role, helping users better understand the relevance of the identified gaps.

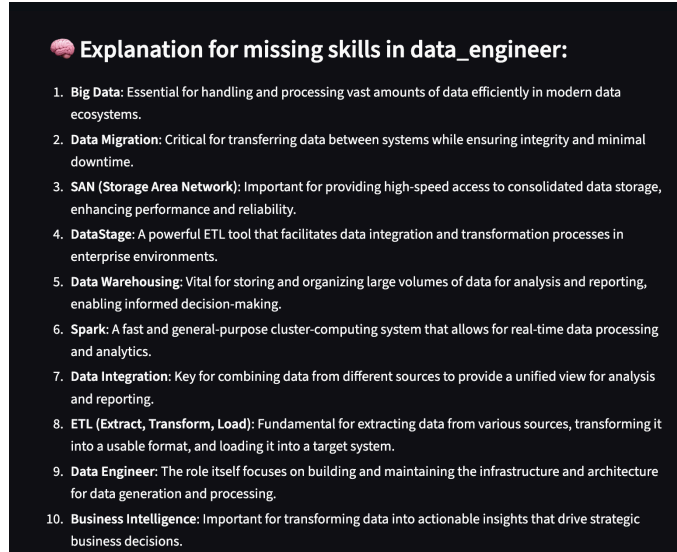


Figure 5: This figure demonstrates the use of a language model to generate concise, human-readable explanations for missing skills.

9. Conclusion and Future Work

This project presented a skill-based job recommendation system that predicts suitable job roles from a given set of skills. By learning associations between skills and job roles from large-scale real-world job posting data, the system generates ranked job role recommendations and identifies potential skill gaps. The combination of TF-IDF-based skill vectorization and a linear classifier trained using stochastic gradient descent proved to be both effective and scalable for this task.

Experimental results demonstrate strong performance across classification and ranking metrics, with particularly high Top-K accuracy. These results indicate that, while predicting a single exact job title remains challenging due to overlap between related roles, a ranking-based approach successfully identifies a small and relevant set of job roles for a given skill profile. This behavior closely aligns with practical resume screening and career guidance scenarios, where multiple suitable roles may exist.

Several opportunities for future work arise from this study. One direction is the incorporation of richer semantic representations, such as contextual word embeddings, to better capture relationships between related skills expressed using different terminology. Expanding the role

taxonomy to support finer-grained distinctions could further improve recommendation specificity, particularly for specialized or emerging roles. Additionally, integrating contextual factors such as experience level, industry domain, and temporal trends in skill demand may enhance the relevance and adaptability of recommendations.

Overall, this work demonstrates that combining lightweight machine learning models with careful data preprocessing and practical tooling can yield an effective and interpretable job recommendation system. The proposed approach provides a solid foundation for future research and development in automated career guidance and intelligent hiring support systems.